

Código

```
require("node:dns/promises").setServers(["0.0.0.0", "8.8.8.8"]);

const express = require("express");
const { MongoClient } = require("mongodb");

const app = express();

let client = null;
let database = null;
let collection = null;

const dbName = "myDatabase";
const collectionName = "recipes";

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

/**
 * Función para conectarse a MongoDB Atlas.
 * Crea una instancia del cliente y establece la conexión.
 */
async function connectDB() {
  const uri =

  "mongodb+srv://Karen_db_user:KarenLVG817381@cluster0.smqcyvr.mongodb.net/
  ?appName=Cluster0";

  client = new MongoClient(uri);

  // Establece la conexión con MongoDB
  await client.connect();

  console.log("Conexión exitosa a MongoDB");
}

/**
 * Función para preparar referencias a la base de datos
 */
function prepareDB() {
  database = client.db(dbName);
  collection = database.collection(collectionName);

  console.log(`Base de datos seleccionada: ${dbName}`);
  console.log(`Colección seleccionada: ${collectionName}`);
}

app.get("/", async function (req, res) {
  const response = {
    message: "Nothing to send",
  };
});
```

```

    res.json(response);
  });

  /**
   * Servicio GET que recibe parámetros por URL.
   * /serv001?id=Nope&token=2345678dhu43567fgh&geo=123456789,1234567890
   */
  app.get("/serv001", async function (req, res) {
    const user_id = req.query.id;
    const token = req.query.token;
    const geo = req.query.geo;

    const response = {
      user_id: user_id,
      token: token,
      geo: geo,
    };

    res.json(response);
  });

  /**
   * Servicio GET.
   * Recibe los parámetros por query string.
   */
  app.get("/serv0010", async function (req, res) {
    const user_id1 = req.query.id;
    const token1 = req.query.token;
    const geo1 = req.query.geo;

    const response = {
      user_id: user_id1,
      token: token1,
      geo: geo1,
    };

    res.json(response);
  });

  /**
   * Servicio POST que recibe datos en el body en formato JSON.
   * {
   *   "id": "nope",
   *   "token": "ertydfg456Dfgwerty",
   *   "geo": "12345678,34567890"
   * }
   */
  app.post("/serv002", async function (req, res) {
    const user_id = req.body.id;
    const token = req.body.token;
    const geo = req.body.geo;

    const response = {

```

```

        user_id: user_id,
        token: token,
        geo: geo,
    };

    res.json(response);
});

/**
 * Servicio POST que recibe un parámetro como parte de la URL.
 * /serv003/1234567
 */
app.post("/serv003/:info", async function (req, res) {
    const info = req.params.info;

    const response = {
        info: info,
    };

    res.json(response);
});

/**
 * Servicio POST para insertar recetas en MongoDB.
 *
 * recibe el arreglo "recipes" desde un JSON en el body.
 *
 * Ejemplo
 * {
 *   "recipes": [
 *     {
 *       "name": "elotes cocidos",
 *       "ingredients": [
 *         "corn",
 *         "mayonnaise",
 *         "cotija cheese",
 *         "sour cream",
 *         "lime"
 *       ],
 *       "prepTimeInMinutes": 35
 *     },
 *     {
 *       "name": "quesadilla",
 *       "ingredients": [
 *         "tortilla",
 *         "queso"
 *       ],
 *       "prepTimeInMinutes": 10
 *     }
 *   ]
 * }
 */
app.post("/receipt/insert", async function (req, res) {

```

```

const recipes = req.body.recipes;

// Validación: verificar que recipes exista y sea un arreglo
if (!recipes || !Array.isArray(recipes) || recipes.length === 0) {
  return res.status(400).json({
    result: "Error: el campo 'recipes' es obligatorio y debe ser un
arreglo con al menos un elemento.",
  });
}

try {
  const insertManyResult = await collection.insertMany(recipes);

  console.log(
    `${insertManyResult.insertedCount} documents successfully
inserted.`
  );

  res.json({
    result: `${insertManyResult.insertedCount} documents successfully
inserted.`,
    insertedIds: insertManyResult.insertedIds,
  });
} catch (err) {
  console.error(
    `Something went wrong trying to insert the new documents: ${err}`
  );

  res.status(500).json({
    result: `Something went wrong trying to insert the new documents:
${err.message}`,
  });
}
});

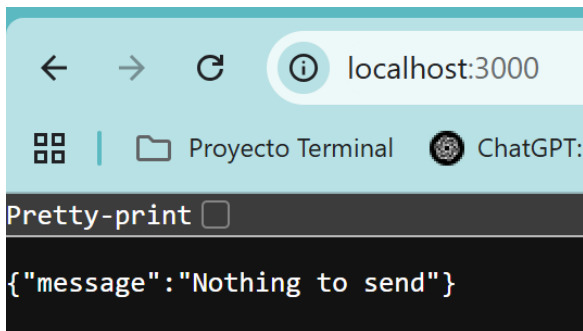
/**
 * Función principal para iniciar el servidor.
 */
async function startServer() {
  try {
    await connectDB();
    prepareDB();

    app.listen(3000, function () {
      console.log("Aplicación ejemplo, escuchando el puerto 3000!");
    });
  } catch (error) {
    console.error("Error al iniciar el servidor:", error);
  }
}

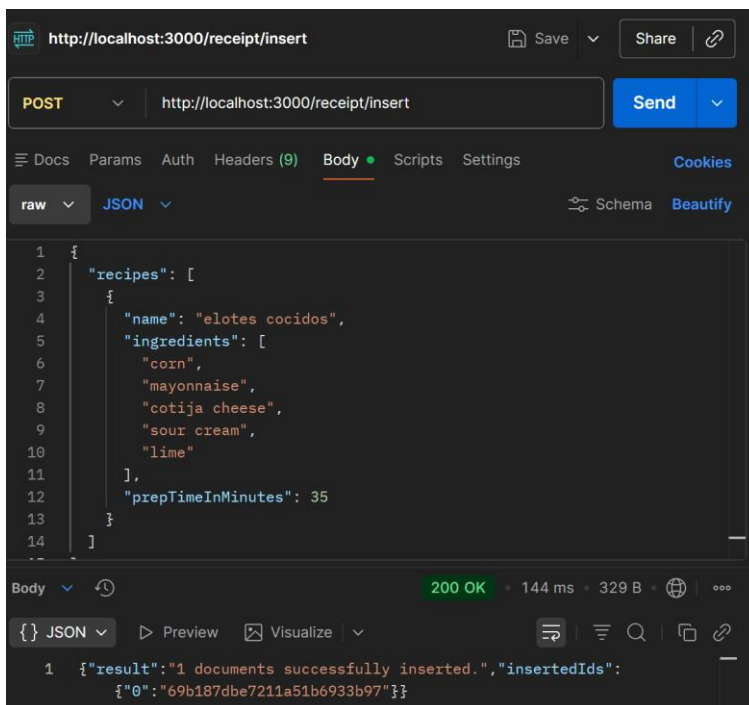
startServer();

```

En localhost



Postman



MongoDB

